



Backend Documentation

Overview

This backend application is built using FastAPI and serves as a RESTful API for handling PDF document uploads, processing, and querying. It utilizes various libraries such as PyMuPDF for PDF processing, llama-index for natural language processing, and SQLite for data storage.

Key Components

1. FastAPI Application

The core of the application is built around a FastAPI instance (`app`).

Role:

- Handles routing for various endpoints
- Manages CORS middleware
- Provides database connection management

2. Database Layer

SQLite database is used for storing uploaded PDF documents and related metadata.

Role:

- Stores document information (filename, content, file_hash, uploaded_date)
- Supports querying and updating document records

3. PDF Processing Module

Utilizes PyMuPDF (fitz library) for extracting text from uploaded PDF files.

Role:

- Extracts text content from PDF documents
- Handles PDF file hashing for duplicate checking

4. Natural Language Processing (NLP) Layer

Llama-index is integrated for natural language processing capabilities.

Role:

- Provides vector embeddings for document content
- Enables semantic search and question answering

5. File Upload System

Handles file uploads and storage.

Role:

- Receives PDF file uploads
- Stores uploaded files in the local filesystem
- Manages file naming and organization

6. Vector Store Indexing

Creates and manages vector store indexes for efficient querying.

Role:

- Builds vector representations of document content
- Supports fast similarity searches

7. Chat Engine Integration

Integrates with a chat engine for generating responses to user queries.

Role:

- Processes user questions
- Generates relevant responses based on document content

Architecture Flow

1. User initiates file upload through the frontend.
2. File is received by the `/upload-pdf/` endpoint.
3. PDF text extraction occurs using PyMuPDF.
4. Document metadata is stored in the SQLite database.
5. Vector store index is created for efficient querying.
6. User can submit questions about uploaded documents.
7. Questions are processed by the chat engine.
8. Responses are generated based on relevant document content.
9. Results are streamed back to the client.

Key Interactions

Database Interaction

- FastAPI application connects to the SQLite database.
- CRUD operations are performed on the `documents` table.

PDF Processing

- PyMuPDF extracts text from uploaded PDF files.
- File hashing occurs to detect duplicates.

Vector Store Indexing

- Llama-index creates vector representations of document content.
- Efficient similarity searches are enabled.

Chat Engine Integration

- User queries are processed by the chat engine.
- Relevant document content is retrieved based on embeddings.

Endpoints

1. `/check-api-key`

- Purpose: Verify the presence of the GROQ API key.

2. `/upload-pdf/`

- Purpose: Upload a PDF file and store its metadata.
- Interaction: Receives file upload, processes PDF, stores metadata, creates index.

3. `/ask-question/`

- Purpose: Process user questions about uploaded documents.
- Interaction: Retrieves relevant document content, generates response.

4. /list-uploads/

- Purpose: List recently uploaded PDF files.
- Interaction: Fetches list of PDF filenames from the uploads folder.

5. /get-document-id/

- Purpose: Retrieve the document_id for a specific filename.
- Interaction: Queries database by file_hash to find corresponding document_id.

6. /books

- Purpose: Get all books (PDF documents) in the system.
- Interaction: Fetches all entries from the `documents` table.

7. /delete-pdf/{document_id}

- Purpose: Delete a specific PDF document and its associated index.
- Interaction: Removes file from filesystem, deletes entry from database, removes index.

8. /get-all-documents

- Purpose: Retrieve all documents stored in the system.
- Interaction: Fetches all entries from the `documents` table.

Environment Variables

- GROQ_API_KEY: Used for authenticating with the Groq LLM service.

File Organization

- `main.py` : Main FastAPI application file.
- `database.db` : SQLite database file.
- `uploads/` : Directory for storing uploaded PDF files.
- `vector_stores/` : Directory for storing vector store indexes.

Dependencies

- fastapi: Web framework
- pydantic: Data validation
- llama-index: Natural language processing library
- fitz: PyMuPDF library for PDF processing
- sqlite3: SQLite database module
- asyncio: Asynchronous programming support
- os: Operating system interface
- json: JSON data manipulation
- shutil: High-level file operations
- hashlib: Secure hash and message digest algorithms
- datetime: Basic date and time types

UI / Frontend Documentation

Overview

The frontend of the PDF Questioner application is built using React.js and serves as the

user interface for interacting with the backend API. It provides features for uploading PDF files, submitting questions, and displaying responses.

Key Components

1. React Application

The core of the frontend is built around a React application instance.

Role:

- Manages component state and lifecycle
- Handles user interactions and events
- Renders UI elements based on application state

2. File Upload Component

Handles file selection and upload functionality.

Role:

- Provides file input interface
- Initiates file upload process
- Displays upload progress

3. Question Submission Form

Allows users to enter questions about uploaded PDF documents.

Role:

- Collects user input for questions
- Validates user input

- Submits questions to the backend

4. Response Display Component

Displays the generated responses from the backend.

Role:

- Receives and displays responses from the backend
- Formats and presents the response content

5. Document List Component

Displays a list of recently uploaded PDF documents.

Role:

- Fetches document list from the backend
- Displays filenames of uploaded documents

6. Chat Interface

Provides a conversational interface for interacting with the AI chat engine.

Role:

- Manages conversation flow
- Displays chat history
- Handles user input submission

Architecture Flow

1. User opens the frontend application.
2. User selects a PDF file to upload through the File Upload component.
3. Backend receives the file and processes it.
4. User submits a question about the uploaded document.
5. Question is sent to the backend for processing.
6. Backend generates a response based on the document content.
7. Response is received by the frontend and displayed.
8. User can interact with the chat interface to ask additional questions.

Key Interactions

Backend Interaction

- React application communicates with the FastAPI backend via HTTP requests.
- Uses Axios for making API calls to the backend endpoints.

State Management

- React State is used for state management.

UI Components

- Tailwind CSS is utilized for consistent styling and layout.
- Custom components are created for specific functionality.

Dependencies

- react: JavaScript library for building user interfaces
- react-dom: DOM renderer for React
- @reduxjs/toolkit: Toolkit for efficient Redux development
- axios: Promise-based HTTP client

- `material-ui/core`: Material Design components
- `@emotion/react`: CSS-in-JS solution
- `@emotion/styled`: Styled-components support
- `react-router-dom`: Router for single-page application navigation
- `react-hook-form`: Form state management library

File Organization

- `src/` : Main source directory
 - `components/` : Directory for individual UI components
 - `pages/` : Directory for main page layouts
 - `store/` : Directory for Redux store setup
 - `utils/` : Utility functions
 - `api/` : API request handlers
 - `App.js` : Main App component
 - `index.js` : Entry point of the React application